LAPORAN PRAKTIKUM

POSTTEST 7

ALGORITMA PEMROGRAMAN DASAR



Disusun oleh:

Rafif Fathi Mohammad 2609106041

Kelas B1 ‘26

PROGRAM STUDI INFORMATIKA

UNIVERSITAS MULAWARMAN

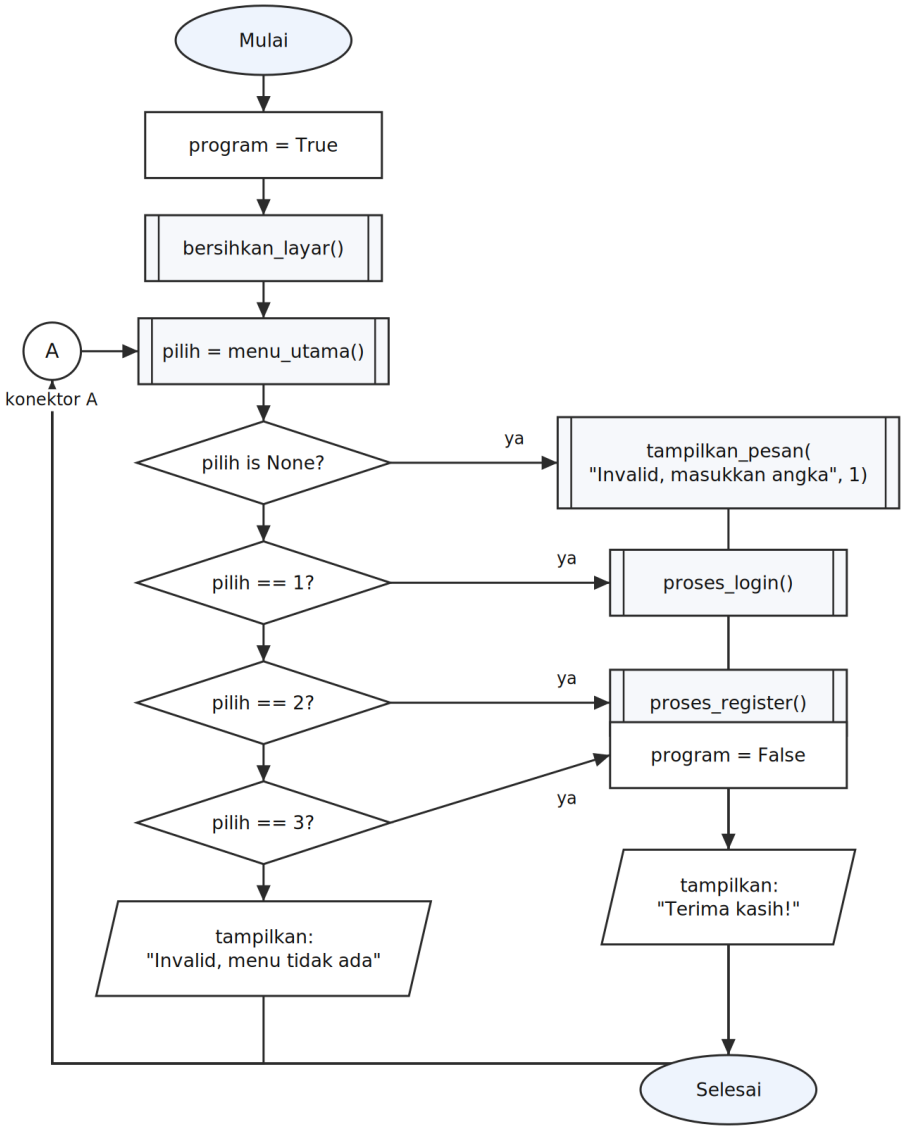
SAMARINDA

2026

1. Flowchart

Seluruh pemanggilan fungsi dan prosedur digambarkan dengan simbol *Predefined Process* (persegi panjang bergaris ganda), sedangkan proses yang terjadi di dalam fungsi tersebut diuraikan pada flowchart tersendiri.

1.1 Menu Utama

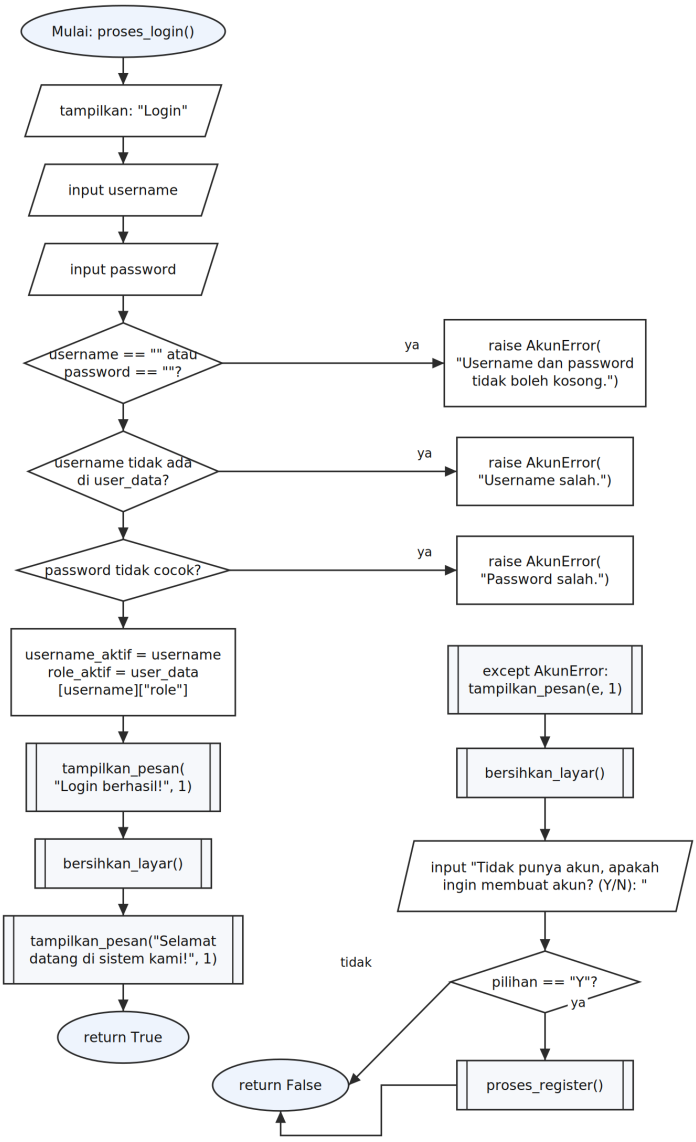


Gambar 1.1 Flowchart Menu Utama (*main*)

Alur program dimulai dari prosedur **main()** yang menginisialisasi *program = True*, membersihkan layar, lalu memanggil fungsi **menu_utama()** (digambarkan sebagai *Predefined Process*). Fungsi tersebut mengembalikan nilai *int* apabila pengguna memasukkan angka, dan mengembalikan *None* apabila masukan bukan angka karena *ValueError* ditangkap di dalam fungsi. Nilai *None* diperiksa lebih dahulu sehingga program menampilkan pesan “Invalid, masukkan angka”. Angka 1 diarahkan ke prosedur **proses_login()**, angka 2 ke **proses_register()**, angka 3 mengubah *program* menjadi *False* dan menampilkan “Terima kasih!”, sedangkan angka lain yang masih berupa bilangan bulat masuk ke

cabang *else* dengan pesan “Invalid, menu tidak ada”. Seluruh cabang yang tidak mengakhiri program dikembalikan melalui konektor A ke pemanggilan *menu_utama()*.

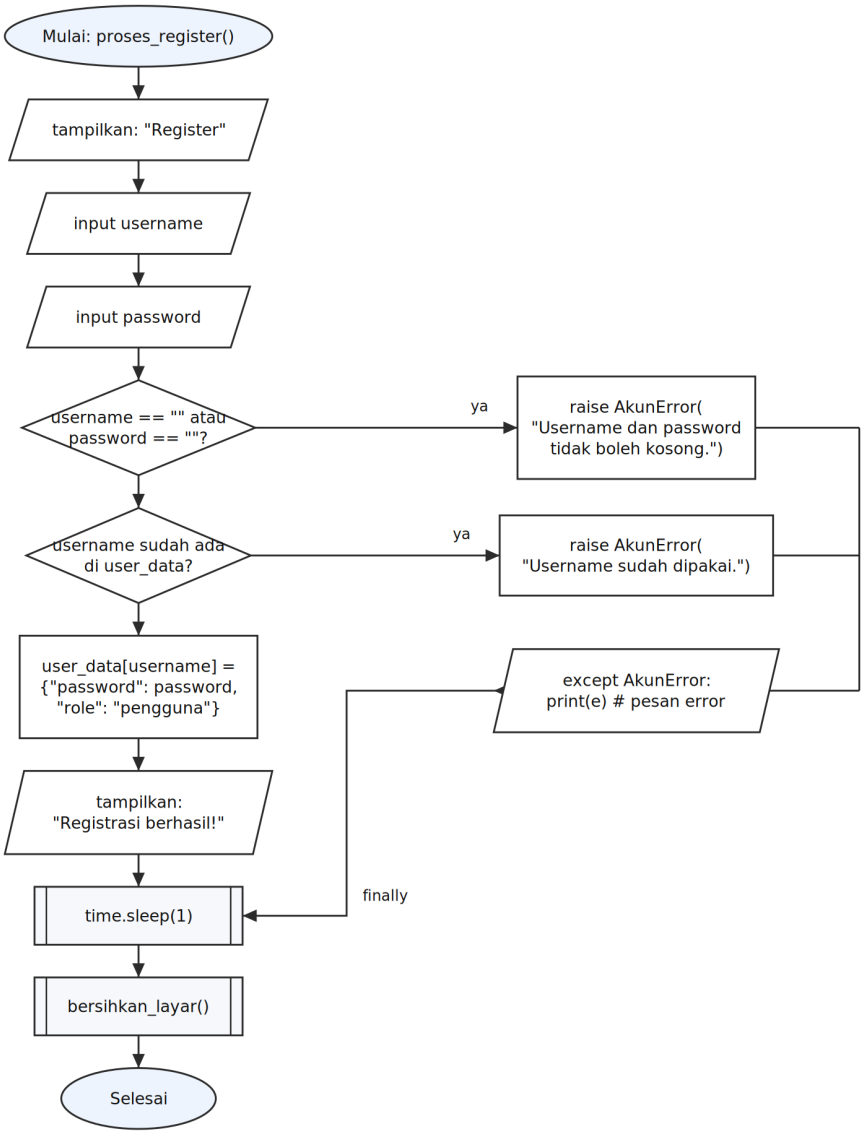
1.2 Login



Gambar 1.2 Flowchart Login (*proses_login*)

Prosedur **proses_login()** meminta *username* dan *password*, kemudian memvalidasi tiga kondisi secara berurutan. Setiap kegagalan tidak langsung mencetak pesan, melainkan melempar objek pengecualian **AkunError** dengan pesan yang berbeda: kolom kosong, username tidak ada di *user_data*, atau password tidak cocok. Blok *except AkunError* menampilkan pesan tersebut, membersihkan layar, lalu menawarkan pendaftaran akun; bila pengguna menjawab “Y” prosedur **proses_register()** dipanggil, dan prosedur tetap mengembalikan nilai *False*. Apabila seluruh validasi lolos, variabel global *username_aktif* dan *role_aktif* diisi, pesan sambutan ditampilkan, dan fungsi mengembalikan nilai *True* sebagai penanda bahwa dashboard boleh dibuka.

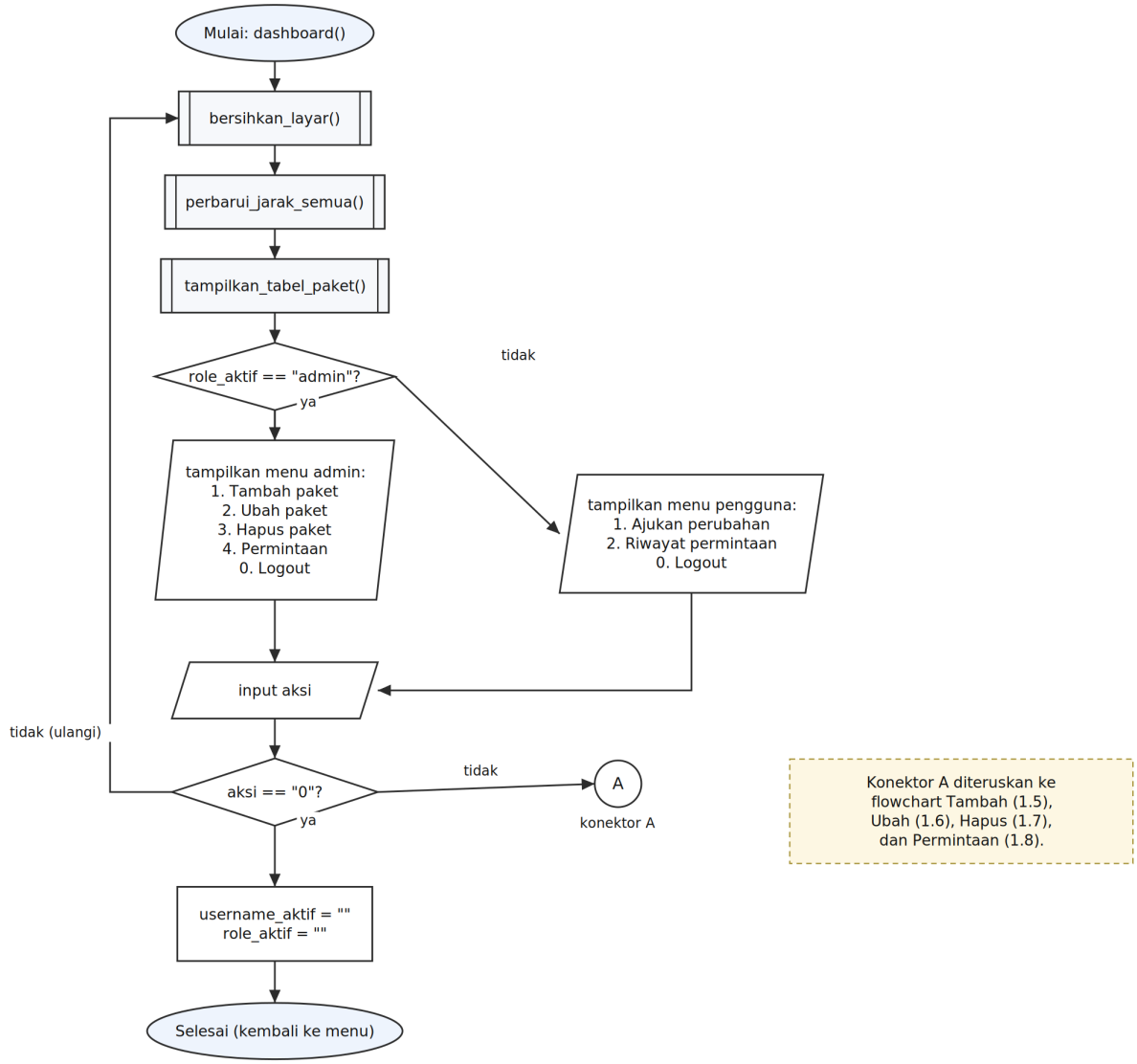
1.3 Register



Gambar 1.3 Flowchart Register (*proses_register*)

Prosedur **proses_register()** meminta *username* dan *password* baru. Dua kondisi diperiksa dengan mekanisme *raise*: kolom kosong dan username yang sudah terpakai, keduanya melempar **AkunError**. Jika lolos, akun baru disimpan ke *user_data* dengan peran “pengguna” dan pesan “Registrasi berhasil!” ditampilkan. Baik jalur berhasil maupun jalur gagal sama-sama melewati blok *finally* yang berisi *time.sleep(1)* dan **bersihkan_layar(), sehingga pesan selalu sempat terbaca sebelum layar dibersihkan.**

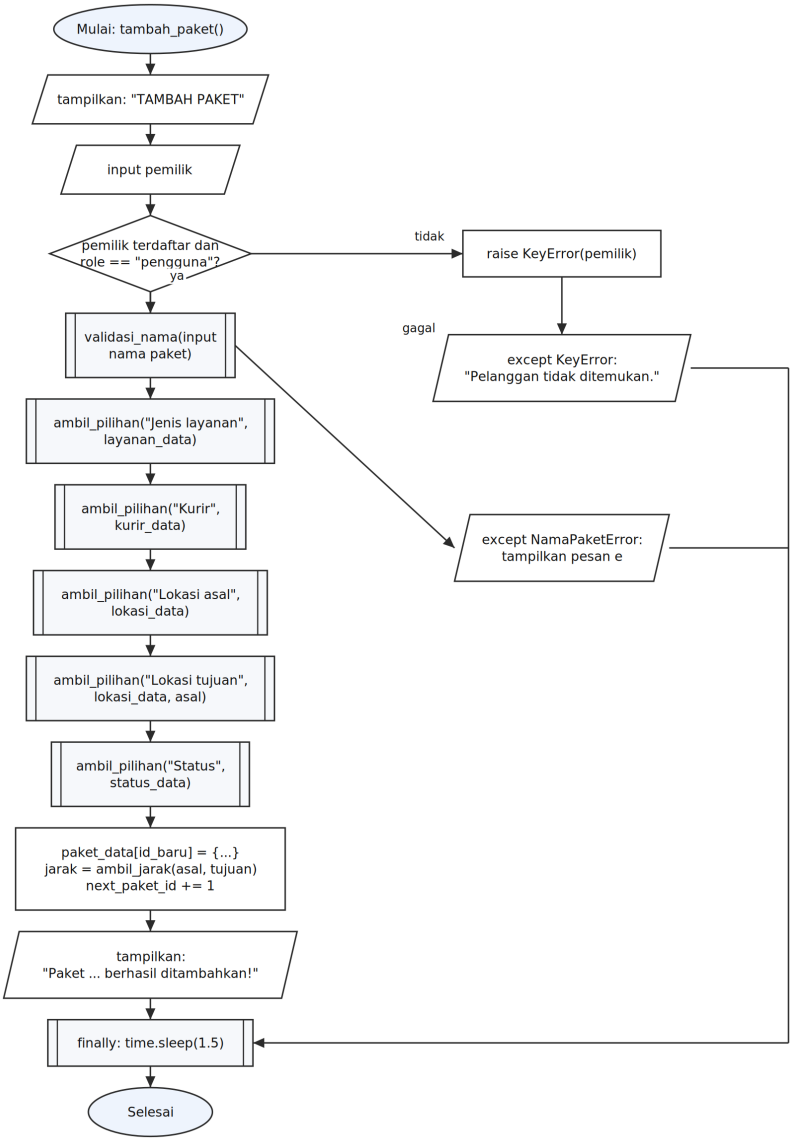
1.4 Read



Gambar 1.4 Flowchart Read (*dashboard*)

Flowchart ini menggambarkan prosedur dashboard() yang menjadi pusat menu setelah login. Setiap perulangan dimulai dengan bersihkan_layar(), perbarui_jarak_semua(), dan tampilkan_tabel_paket() (ketiganya *Predefined Process*). Isi menu bergantung pada *role_aktif*: admin memperoleh empat opsi pengelolaan, pengguna hanya memperoleh pengajuan perubahan dan riwayat permintaan. Masukan *aksi* bernilai "0" akan mengosongkan *username_aktif* dan *role_aktif* lalu keluar; selain itu alur diteruskan melalui konektor A ke flowchart Tambah, Ubah, Hapus, atau Permintaan, dan kembali ke awal perulangan.

1.5 Tambah



Gambar 1.5 Flowchart Tambah (*tambah_paket*)

Prosedur tambah_paket() meminta username pelanggan lebih dahulu. Bila username tidak terdaftar atau perannya bukan “pengguna”, program melempar KeyError yang ditangkap dan diterjemahkan menjadi pesan “Pelanggan tidak ditemukan.” Nama paket divalidasi oleh fungsi validasi_nama() yang melempar NamaPaketError bila kosong atau melebihi 15 karakter. Lima data berikutnya diambil melalui fungsi ambil_pilihan(), termasuk pemanggilan dengan argumen ketiga berupa lokasi asal agar tujuan tidak boleh sama. Apabila semua data valid, blok *else* menyimpan paket ke *paket_data* beserta jarak hasil ambil_jarak() dan menaikkan *next_paket_id*. Blok *finally* selalu menjalankan *time.sleep(1.5)* pada semua jalur.

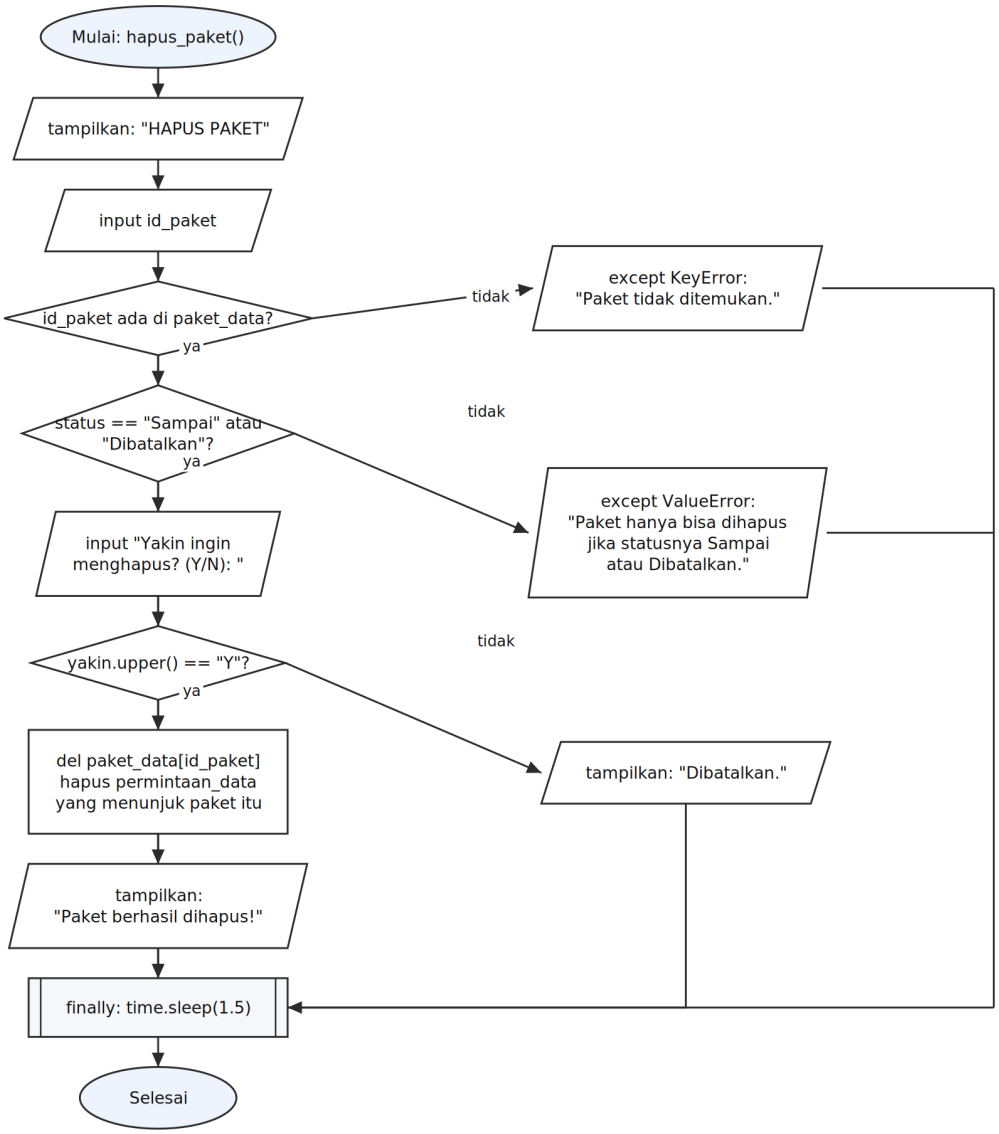
1.6 Ubah



Gambar 1.6 Flowchart Ubah (*ubah_paket*)

Prosedur ubah_paket() dipakai admin untuk mengubah data dan oleh pengguna untuk mengajukan permintaan. Fungsi cari_paket_milik() memastikan paket ada dan boleh diakses; kegagalannya berupa KeyError. Status “Sampai” atau “Dibatalkan” ditolak melalui *ValueError*. Nomor field dibaca dengan *int(input(...))* di dalam blok *try* tersendiri sehingga masukan bukan angka langsung ditangani dan prosedur berhenti tanpa menghentikan program, sementara nomor di luar rentang ditangani sebagai IndexError. Nama paket divalidasi validasi_nama(), field lain melalui ambil_pilihan(), lalu asal dan tujuan dibandingkan (LokasiSamaError) dan nilai baru dibandingkan dengan nilai lama (*ValueError*). Jika peran admin, data langsung diperbarui dan jarak dihitung ulang; jika peran pengguna, dibuat entri *permintaan_data* berstatus “Menunggu”.

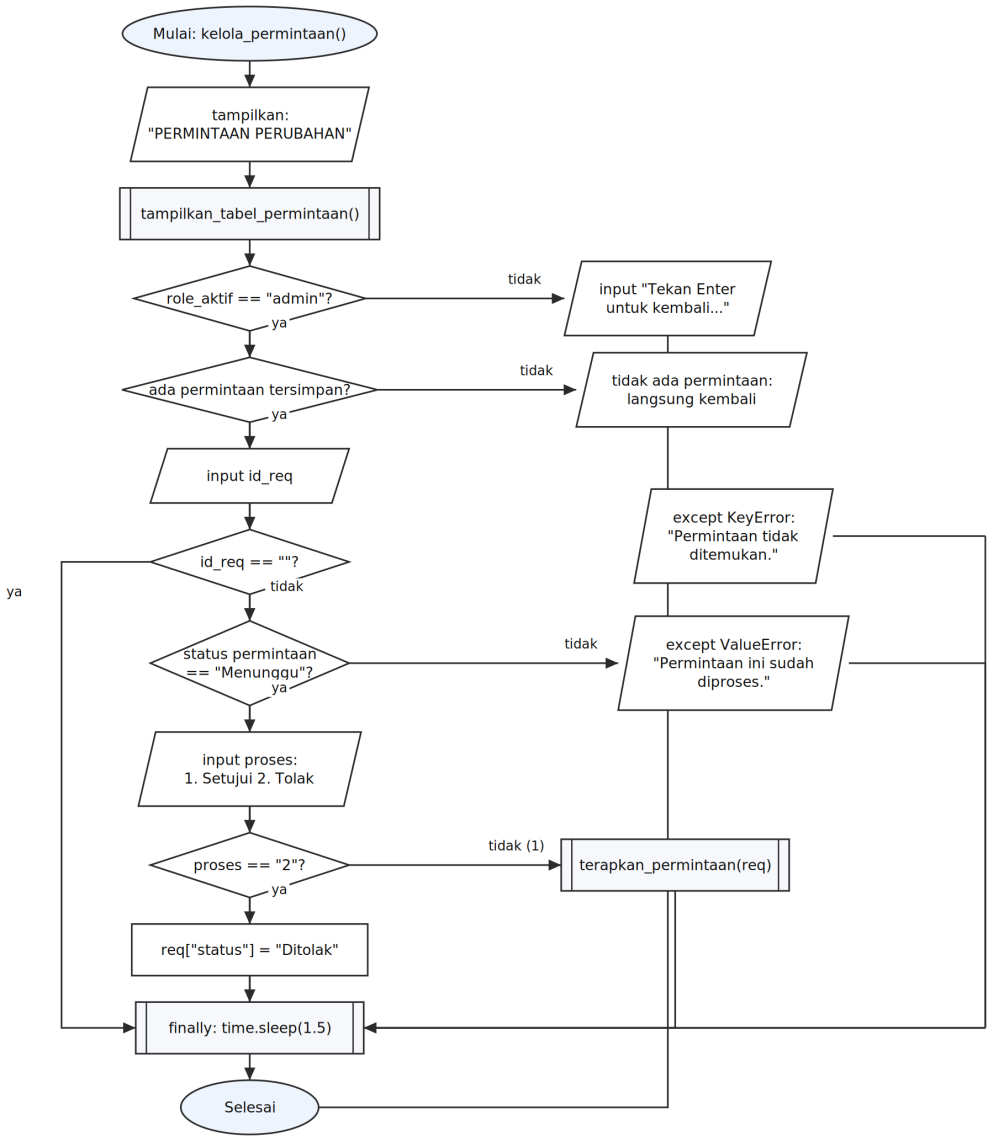
1.7 Hapus



Gambar 1.7 Flowchart Hapus (*hapus_paket*)

Prosedur hapus_paket() hanya dapat dijalankan admin. ID paket yang tidak ditemukan ditangani sebagai KeyError, sedangkan paket yang statusnya belum “Sampai” atau “Dibatalkan” ditolak dengan *ValueError* agar data yang masih dalam proses tidak hilang. Setelah pengguna menjawab “Y” pada konfirmasi, paket dihapus dari *paket_data* dan seluruh permintaan yang menunjuk paket tersebut ikut dihapus. Jawaban selain “Y” hanya menampilkan pesan “Dibatalkan.” Semua jalur tetap melewati *finally* berisi *time.sleep(1.5)*.

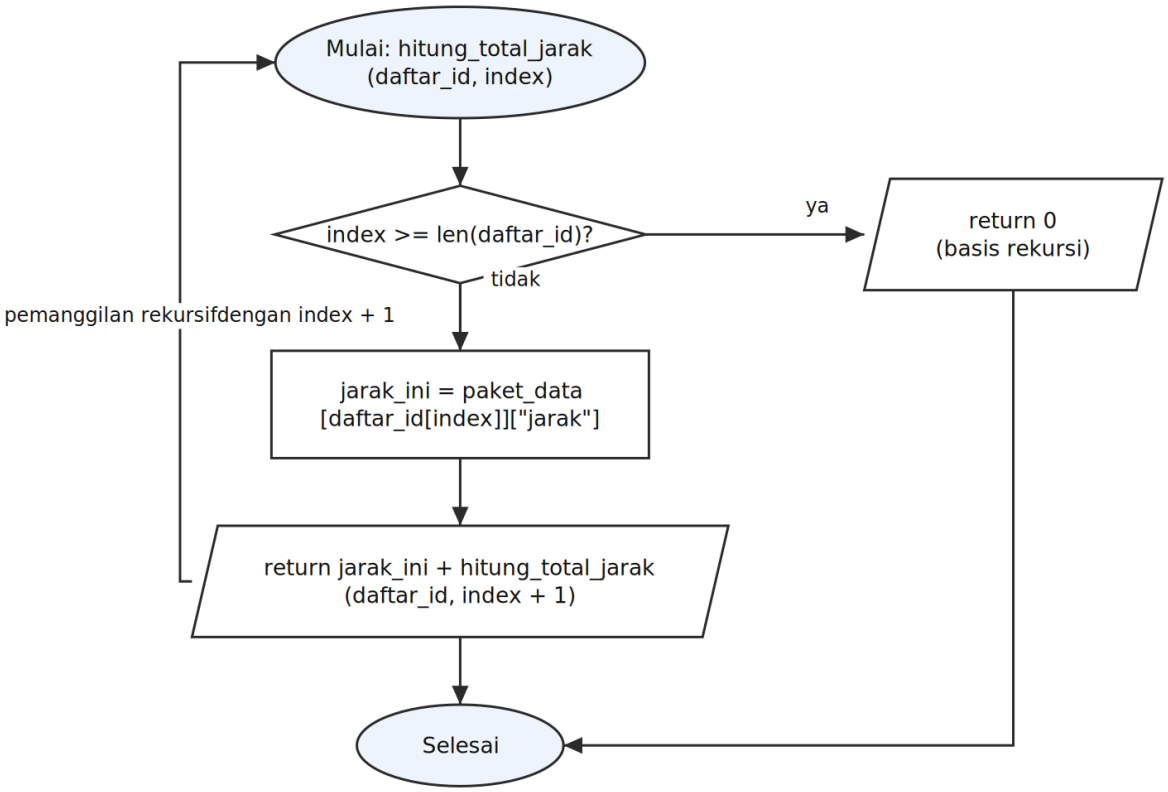
1.8 Permintaan



Gambar 1.8 Flowchart Permintaan (*kelola_permintaan*)

Prosedur kelola_permintaan() menampilkan tabel permintaan lebih dahulu. Bagi pengguna, alur berhenti setelah menekan Enter karena hanya bersifat riwayat. Bagi admin, ID permintaan diminta; ID yang tidak ada ditangani sebagai KeyError dan permintaan yang sudah diproses ditolak dengan *ValueError*. Pilihan "2" langsung mengubah status menjadi "Ditolak", sedangkan pilihan "1" memanggil prosedur terapkan_permintaan() yang memeriksa ulang keberadaan paket, status paket, serta kesamaan asal dan tujuan sebelum menulis nilai baru dan menghitung ulang jarak.

1.9 Rekursi



Gambar 1.9 Flowchart Rekursi (*hitung_total_jarak*)

Fungsi hitung_total_jarak(daftar_id, index) menghitung total jarak seluruh paket yang tampil pada tabel. Kasus dasar terjadi ketika *index* sudah mencapai *len(daftar_id)*; pada kondisi itu fungsi mengembalikan nilai 0 sehingga penjumlahan berhenti. Selama *index* masih valid, fungsi mengambil jarak paket ke-*index* lalu menambahkan hasil pemanggilan dirinya sendiri dengan *index + 1*. Pola ini membuat total dihitung satu per satu secara rekursif, dan hasilnya ditampilkan pada baris "Total jarak seluruh paket" di bawah tabel.

2. Deskripsi Singkat Program

Program ini adalah Sistem Manajemen Pengiriman Paket berbasis teks yang dijalankan di terminal. Program menyediakan autentikasi melalui Login dan Register, lalu pengelolaan paket yang mencakup penambahan, penampilan, perubahan, dan penghapusan data. Hak akses dibedakan antara *admin* dan *pengguna*: admin dapat melihat seluruh paket serta mengelola datanya, sedangkan pengguna hanya melihat paket miliknya dan dapat mengajukan permintaan perubahan yang harus disetujui admin. Jarak tempuh tidak dimasukkan manual, melainkan diambil dari tabel *jarak_data* berdasarkan pasangan lokasi asal dan tujuan, kemudian dijumlahkan secara rekursif untuk ditampilkan sebagai total jarak seluruh paket.

Dibandingkan Posttest 6, struktur program dipecah menjadi banyak fungsi dan prosedur, penanganan kesalahan dipindahkan dari pemeriksaan *if* bertingkat ke mekanisme *try*/*except*/*else*/*finally* dengan tiga kelas pengecualian buatan sendiri, serta ditambahkan satu fungsi rekursif.

2.1 Fungsi dan Prosedur

Prosedur adalah subprogram yang bekerja langsung pada variabel global atau variabel di dalam fungsinya dan tidak mengembalikan nilai, sedangkan fungsi mengembalikan nilai melalui *return*. Tabel berikut memuat seluruh subprogram beserta parameter dan nilai kembaliannya sesuai kode program.

Nama subprogram	Jenis	Parameter	Return	Keterangan
`bersihkan_layar`	prosedur	–	–	Menjalankan *os.system(clear)*; nilai *clear* berisi “cls” pada Windows dan “clear” pada Unix.
`tampilkan_pesan`	prosedur	pesan, detik	–	Mencetak pesan lalu *time.sleep(detik)* agar pesan sempat terbaca.
`tampilkan_tabel_paket`	prosedur	–	–	Menyusun tabel dengan *ljust*, memakai ambil_id_terlihat() dan menutup tabel dengan total jarak.
`tampilkan_tabel_permintaan`	prosedur	–	–	Menampilkan permintaan milik pengguna atau seluruh permintaan bila admin.
`perbarui_jarak_semua`	prosedur	–	–	Menghitung ulang *jarak* setiap paket; *KeyError* membuat jarak bernilai 0 disertai peringatan.
`proses_register`	prosedur	–	–	Menambah akun baru; memakai AkunError dan blok *finally*.
`tambah_paket`	prosedur	–	–	Menambah paket; memakai *global next_paket_id* serta blok *else* dan *finally*.
`ubah_paket`	prosedur	–	–	Mengubah paket (admin) atau membuat permintaan (pengguna); *global next_req_id*.

Nama subprogram	Jenis	Parameter	Return	Keterangan
`hapus_paket`	prosedur	–	–	Menghapus paket beserta permintaan yang terkait setelah konfirmasi.
`kelola_permintaan`	prosedur	–	–	Menampilkan dan memproses permintaan perubahan.
`terapkan_permintaan`	prosedur	req	–	Menerapkan nilai baru dari satu permintaan setelah validasi ulang.
`dashboard`	prosedur	–	–	Perulangan menu utama setelah login; memakai *global* *username_aktif*, *role_aktif*.
`menu_utama`	fungsi	–	int / None	Mengembalikan nomor menu, atau *None* bila masukan bukan angka.
`ambil_id_terlihat`	fungsi	–	list	Mengumpulkan ID paket yang boleh dilihat sesuai peran.
`ambil_jarak`	fungsi	asal, tujuan	int	Mengambil jarak dari *jarak_data[asal][tujuan]*.
`validasi_nama`	fungsi	nama	str	Melempar NamaPaketError bila kosong atau lebih dari 15 karakter.
`cari_paket_milik`	fungsi	id_paket	dict	Mengembalikan data paket; *KeyError* bila tidak ada atau bukan milik pengguna.
`ambil_pilihan`	fungsi	judul, opsi, dilarang	str	Perulangan *input* sampai valid; parameter *dilarang* menolak tujuan yang sama dengan asal.
`hitung_total_jarak`	fungsi (rekursif)	daftar_id, index	int	Menjumlahkan jarak seluruh paket secara rekursif.
`proses_login`	fungsi	–	bool	Mengembalikan *True* bila login berhasil dan *False* bila gagal.
`main`	prosedur	–	–	Perulangan menu awal sampai pengguna memilih Keluar.

2.2 Variabel Global dan Lokal

Variabel global dideklarasikan di bagian atas berkas dan dapat dibaca oleh semua subprogram: `user_data`, `layanan_data`, `kurir_data`, `lokasi_data`, `status_data`, `jarak_data`, `paket_data`, `permintaan_data`, `edit_fields`, `next_paket_id`, `next_req_id`, `username_aktif`, `role_aktif`, dan `clear`. Karena isinya berupa objek *mutable* (dictionary dan list), perubahan elemen di dalamnya tidak memerlukan kata kunci *global*. Kata kunci *global* hanya dipakai ketika variabel global diberi nilai baru, yaitu pada `tambah_paket` (`next_paket_id`), `ubah_paket` (`next_req_id`), serta `proses_login` dan `dashboard` (`username_aktif` dan `role_aktif`).

Variabel lokal hanya hidup di dalam subprogram tempat ia dibuat, misalnya `pilih` pada `main`, `username` dan `password` pada `proses_login`, `target` dan `baru` pada `ubah_paket`, `daftar_id` dan `baris` pada `tampilkan_tabel_paket`, serta parameter `daftar_id` dan `index` pada `hitung_total_jarak`. Nilai yang harus dipakai lintas subprogram dikembalikan melalui *return* (misalnya daftar ID dari `ambil_id_terlihat`) atau disimpan pada variabel global (misalnya `role_aktif`).

2.3 Penanganan Kesalahan (Error Handling)

Tiga kelas pengecualian dibuat sendiri dengan mewarisi `Exception`, yaitu NamaPaketError, LokasiSamaError, dan AkunError. Kelas buatan sendiri dipakai agar kesalahan yang berbeda dapat ditangkap oleh blok *except* yang berbeda, sehingga pesan yang muncul selalu sesuai penyebabnya. Pengecualian bawaan Python (*ValueError*, *KeyError*, *IndexError*) tetap dimanfaatkan untuk kasus yang memang cocok, misalnya *ValueError* dari `int(input(...))` dan *KeyError* ketika kunci dictionary tidak ditemukan.

Pengecualian	Penyebab	Penanganan
NamaPaketError	Nama paket kosong atau lebih dari 15 karakter (fungsi `validasi_nama`).	Ditangkap di `tambah_paket` dan `ubah_paket`; pesan dicetak dan data tidak disimpan.
LokasiSamaError	Lokasi tujuan sama dengan lokasi asal.	Ditangkap di `ambil_pilihan` (perulangan dimulai ulang), `ubah_paket`, dan `terapkan_permintaan`.
AkunError	Kolom login/register kosong, username tidak terdaftar, atau password salah.	Ditangkap di `proses_login` dan `proses_register`; pesan dicetak melalui `tampilkan_pesan` atau `print(e)`.
KeyError	Pelanggan tidak ada, paket/ID permintaan tidak ditemukan, atau pasangan jarak tidak tersedia.	Ditangkap di hampir semua prosedur dan diterjemahkan menjadi pesan “... tidak ditemukan.”; pada `perbarui_jarak_semua` jarak diisi 0 disertai peringatan.
ValueError	Paket berstatus “Sampai”/“Dibatalkan”, nilai baru sama dengan nilai lama, permintaan sudah diproses, atau `int(input(...))` gagal.	Ditangkap di `ubah_paket`, `hapus_paket`, `kelola_permintaan`, dan `terapkan_permintaan`.
IndexError	Nomor pilihan di luar rentang daftar opsi.	Ditangkap di `ambil_pilihan` dan `ubah_paket` dengan pesan “Pilihan tidak valid.”
else	Tidak ada kesalahan pada blok *try*.	Dipakai di `tambah_paket` untuk menyimpan paket baru hanya ketika seluruh validasi lolos.
finally	Selalu dijalankan, baik berhasil maupun gagal.	Berisi `time.sleep(1.5)` (dan `time.sleep(1)` pada register) agar pesan tetap terbaca.

Pengecualian	Penyebab	Penanganan
KeyboardInterrupt / EOFError	Program dihentikan paksa atau masukan berakhir.	Ditangkap pada blok terluar di bawah fungsi `main`; ditutup dengan `finally` yang mencetak “Sesi program selesai.”

Satu hal yang perlu diperhatikan pada `ubah_paket`: pembacaan nomor field memakai blok *try*/*except ValueError* tersendiri yang di dalamnya terdapat `return`. Walaupun fungsi berhenti pada titik itu, blok *finally* milik *try* terluar tetap dijalankan sehingga `time.sleep(1.5)` tidak terlewat. Dengan cara ini masukan bukan angka tidak lagi dilempar sebagai *ValueError* baru dari dalam blok *except*, seperti pada versi sebelumnya.

2.4 Rekursi

Fungsi hitung_total_jarak(daftar_id, index) adalah satu-satunya fungsi rekursif pada program ini. Fungsi menerima daftar ID yang sedang tampil dan posisi pembacaan. Bila `index` sudah sama atau lebih besar dari panjang daftar, fungsi mengembalikan 0 sebagai *base case*. Selain itu fungsi membaca jarak paket pada posisi tersebut lalu mengembalikan `jarak_ini + hitung_total_jarak(daftar_id, index + 1)`. Karena setiap pemanggilan menaikkan `index` sebesar satu, rekursi dijamin berhenti. Hasilnya dipakai pada baris terakhir `tampilkan_tabel_paket`, yaitu `Total jarak seluruh paket: {hitung_total_jarak(daftar_id, 0)} km`, sehingga total otomatis mengikuti paket yang sedang terlihat oleh pengguna.

3. Source Code

Kode program lengkap tersimpan pada berkas `2609106041-Rafif_Fathi_Mohammad-PT-7.py`. Bagian berikut dikutip langsung dari berkas tersebut.

A. Import dan Variabel Global

```
import os
import time

# ==================== VARIABEL GLOBAL ====================

user_data = {
    "admin":    {"password": "admin1",    "role": "admin"},
    "pengguna": {"password": "pengguna1", "role": "pengguna"}
}

layanan_data = ["Reguler", "Express", "Same Day"]
kurir_data = ["Budi", "Sinta", "Andi"]
lokasi_data = ["Balikpapan", "Samarinda", "Bontang", "Tenggarong", "Penajam"]
status_data = ["Diproses", "Dikirim", "Sampai", "Dibatalkan"]

jarak_data = {
    "Balikpapan": {"Samarinda": 115, "Bontang": 235, "Tenggarong": 130, "Penajam": 40},
    "Samarinda":  {"Balikpapan": 115, "Bontang": 120, "Tenggarong": 35, "Penajam": 150},
    "Bontang":    {"Balikpapan": 235, "Samarinda": 120, "Tenggarong": 155, "Penajam": 275},
    "Tenggarong": {"Balikpapan": 130, "Samarinda": 35, "Bontang": 155, "Penajam": 165},
    "Penajam":    {"Balikpapan": 40, "Samarinda": 150, "Bontang": 275, "Tenggarong": 165}
}

paket_data = {
    "1": {"pemilik": "pengguna", "nama": "Laptop", "layanan": "Express",
          "kurir": "Budi", "asal": "Balikpapan", "tujuan": "Samarinda",
          "jarak": 0, "status": "Dikirim"}
}

permintaan_data = {}

edit_fields = [
    ["Nama paket", "nama", None],
    ["Jenis layanan", "layanan", layanan_data],
    ["Kurir", "kurir", kurir_data],
    ["Lokasi asal", "asal", lokasi_data],
    ["Lokasi tujuan", "tujuan", lokasi_data],
    ["Status", "status", status_data]
]

next_paket_id = 2
next_req_id = 1
username_aktif = ""
role_aktif = ""
clear = "cls" if os.name == "nt" else "clear"
```

B. Kelas Pengecualian Buatan Sendiri

```
# ==================== ERROR (EXCEPTION) KUSTOM ====================

class NamaPaketError(Exception):
    """Dilempar jika nama paket kosong atau lebih dari 15 karakter."""
    pass


class LokasiSamaError(Exception):
    """Dilempar jika lokasi asal sama dengan lokasi tujuan."""
```

```
    pass


class AkunError(Exception):
    """Dilempar jika data login / register tidak valid."""
    pass
```

C. Prosedur Dasar dan Register

```
# ==================== PROSEDUR (tanpa return) ====================

def bersihkan_layar():
    os.system(clear)


def tampilkan_pesan(pesan, detik):
    print(pesan)
    time.sleep(detik)


def tampilkan_tabel_paket():
    header = ["ID", "Pemilik", "Paket", "Layanan", "Kurir", "Asal", "Tujuan", "Jarak", "Status"]
    kolom = ["pemilik", "nama", "layanan", "kurir", "asal", "tujuan", "jarak", "status"]
    widths = [8, 10, 16, 10, 8, 12, 12, 9, 11]

    print(f"Login sebagai: {username_aktif} ({role_aktif})")
    print()
    print("DAFTAR SEMUA PAKET" if role_aktif == "admin" else "PAKET SAYA")

    baris = ""
    for i in range(len(header)):
        baris += header[i].ljust(widths[i])
    print(baris)
    print("-" * sum(widths))

    daftar_id = ambil_id_terlihat()
    for id_p in daftar_id:
        paket = paket_data[id_p]
        baris = id_p.ljust(widths[0])
        for i in range(len(kolom)):
            nilai = str(paket["jarak"]) + " km" if kolom[i] == "jarak" else str(paket[kolom[i]])
            baris += nilai.ljust(widths[i + 1])
        print(baris)

    if len(daftar_id) == 0:
        print("(belum ada paket)")
    else:
        print("-" * sum(widths))
        print(f"Total jarak seluruh paket: {hitung_total_jarak(daftar_id, 0)} km")


def tampilkan_tabel_permintaan():
    header = ["ID", "Pelanggan", "Paket", "Perubahan", "Nilai baru", "Status"]
    kolom = ["pelanggan", "paket", "perubahan", "nilai_baru", "status"]
    widths = [8, 12, 9, 16, 14, 10]

    baris = ""
    for i in range(len(header)):
        baris += header[i].ljust(widths[i])
    print(baris)
    print("-" * sum(widths))

    jumlah = 0
    for id_r in permintaan_data:
        r = permintaan_data[id_r]
        if role_aktif == "admin" or r["pelanggan"] == username_aktif:
            baris = id_r.ljust(widths[0])
            for i in range(len(kolom)):
```

```
                baris += str(r[kolom[i]]).ljust(widths[i + 1])
            print(baris)
            jumlah += 1

    if jumlah == 0:
        print("(belum ada permintaan)")
    print()


def perbarui_jarak_semua():
    for id_p in paket_data:
        try:
            paket_data[id_p]["jarak"] = ambil_jarak(paket_data[id_p]["asal"],
                                                    paket_data[id_p]["tujuan"])
        except KeyError:
            paket_data[id_p]["jarak"] = 0
            print(f"Peringatan: data jarak paket {id_p} tidak ditemukan.")


def proses_register():
    print("Register")
    username = input("Masukkan username: ").strip()
    password = input("Masukkan password: ").strip()
    try:
        if username == "" or password == "":
            raise AkunError("Username dan password tidak boleh kosong.")
        if username in user_data:
            raise AkunError("Username sudah dipakai.")
        user_data[username] = {"password": password, "role": "pengguna"}
        print("Registrasi berhasil!")
    except AkunError as e:
        print(e)
    finally:
        time.sleep(1)
        bersihkan_layar()
```

D. Prosedur Pengelolaan Data (Tambah, Ubah, Hapus, Permintaan)

```
def tambah_paket():
    global next_paket_id
    print("TAMBAH PAKET")
    pemilik = input("Username pelanggan: ").strip()
    try:
        if pemilik not in user_data or user_data[pemilik]["role"] != "pengguna":
            raise KeyError(pemilik)
        nama = validasi_nama(input("Nama paket (maks 15 karakter): ").strip())
        layanan = ambil_pilihan("Jenis layanan", layanan_data)
        kurir = ambil_pilihan("Kurir", kurir_data)
        asal = ambil_pilihan("Lokasi asal", lokasi_data)
        tujuan = ambil_pilihan("Lokasi tujuan", lokasi_data, asal)
        status = ambil_pilihan("Status", status_data)
    except KeyError:
        print("Pelanggan tidak ditemukan.")
    except NamaPaketError as e:
        print(e)
    else:
        id_baru = str(next_paket_id)
        paket_data[id_baru] = {"pemilik": pemilik, "nama": nama, "layanan": layanan,
                               "kurir": kurir, "asal": asal, "tujuan": tujuan,
                               "jarak": ambil_jarak(asal, tujuan), "status": status}
        next_paket_id += 1
        print(f"Paket {id_baru} berhasil ditambahkan!")
    finally:
        time.sleep(1.5)


def ubah_paket():
    global next_req_id
```

```
    print("UBAH PAKET" if role_aktif == "admin" else "AJUKAN PERUBAHAN PAKET")
    id_paket = input("Masukkan ID paket: ").strip().upper()

    try:
        target = cari_paket_milik(id_paket)
        if target["status"] in ("Sampai", "Dibatalkan"):
            raise ValueError(f"Paket sudah {target['status']}, tidak bisa diubah.")

        print("Data yang ingin diubah:")
        for n, editf in enumerate(edit_fields, 1):
            print(f"  {n}. {editf[0]}")
        try:
            nomor = int(input("Pilih nomor: "))
        except ValueError:
            print("Input harus berupa angka.")
            return
        if nomor < 1:
            raise IndexError
        editf = edit_fields[nomor - 1]
        if editf[2] is None:
            baru = validasi_nama(input("Nama paket baru (maks 15 karakter): ").strip())
        else:
            baru = ambil_pilihan(editf[0] + " baru", editf[2])

        asal_baru = baru if editf[1] == "asal" else target["asal"]
        tujuan_baru = baru if editf[1] == "tujuan" else target["tujuan"]
        if asal_baru == tujuan_baru:
            raise LokasiSamaError("Lokasi asal dan tujuan tidak boleh sama.")
        if baru == target[editf[1]]:
            raise ValueError("Nilai baru sama dengan nilai sekarang.")

        if role_aktif == "admin":
            target[editf[1]] = baru
            perbarui_jarak_semua()
            print("Paket berhasil diubah! Jarak diperbarui otomatis.")
        else:
            id_req = f"REQ{next_req_id:03d}"
            permintaan_data[id_req] = {"pelanggan": username_aktif, "paket": id_paket,
                                       "perubahan": editf[0], "nilai_baru": baru,
                                       "status": "Menunggu"}
            next_req_id += 1
            print("Permintaan perubahan dikirim ke admin.")
    except KeyError:
        print("Paket tidak ditemukan.")
    except (NamaPaketError, LokasiSamaError) as e:
        print(e)
    except IndexError:
        print("Pilihan tidak valid.")
    except ValueError as e:
        print(e)
    finally:
        time.sleep(1.5)


def hapus_paket():
    print("HAPUS PAKET")
    id_paket = input("Masukkan ID paket: ").strip().upper()
    try:
        if id_paket not in paket_data:
            raise KeyError(id_paket)
        if paket_data[id_paket]["status"] not in ("Sampai", "Dibatalkan"):
            raise ValueError("Paket hanya bisa dihapus jika statusnya Sampai atau Dibatalkan.")

        yakin = input("Yakin ingin menghapus? (Y/N): ")
        if yakin.upper() == "Y":
            del paket_data[id_paket]
            id_hapus = [i for i in permintaan_data if permintaan_data[i]["paket"] == id_paket]
            for id_r in id_hapus:
                del permintaan_data[id_r]
            print("Paket berhasil dihapus!")
        else:
```

```
            print("Dibatalkan.")
    except KeyError:
        print("Paket tidak ditemukan.")
    except ValueError as e:
        print(e)
    finally:
        time.sleep(1.5)


def kelola_permintaan():
    print("PERMINTAAN PERUBAHAN")
    tampilkan_tabel_permintaan()

    if role_aktif != "admin":
        input("Tekan Enter untuk kembali...")
        return
    if len(permintaan_data) == 0:
        time.sleep(1.5)
        return

    id_req = input("ID permintaan yang diproses (kosongkan untuk kembali): ").strip().upper()
    if id_req == "":
        return

    try:
        req = permintaan_data[id_req]
        if req["status"] != "Menunggu":
            raise ValueError("Permintaan ini sudah diproses.")

        proses = input("1. Setujui  2. Tolak : ")
        if proses == "2":
            req["status"] = "Ditolak"
            print("Permintaan ditolak.")
        elif proses == "1":
            terapkan_permintaan(req)
        else:
            print("Pilihan tidak valid.")
    except KeyError:
        print("Permintaan tidak ditemukan.")
    except ValueError as e:
        print(e)
    finally:
        time.sleep(1.5)


def terapkan_permintaan(req):
    kunci = ""
    for editf in edit_fields:
        if editf[0] == req["perubahan"]:
            kunci = editf[1]

    try:
        target = paket_data[req["paket"]]
        if target["status"] in ("Sampai", "Dibatalkan"):
            raise ValueError("Paket sudah tidak bisa diubah. Permintaan ditolak.")

        asal_baru = req["nilai_baru"] if kunci == "asal" else target["asal"]
        tujuan_baru = req["nilai_baru"] if kunci == "tujuan" else target["tujuan"]
        if asal_baru == tujuan_baru:
            raise LokasiSamaError("Lokasi asal dan tujuan sama. Permintaan ditolak.")

        target[kunci] = req["nilai_baru"]
        req["status"] = "Disetujui"
        perbarui_jarak_semua()
        print("Permintaan disetujui! Jarak diperbarui otomatis.")
    except KeyError:
        req["status"] = "Ditolak"
        print("Paket sudah tidak ada. Permintaan ditolak.")
    except (ValueError, LokasiSamaError) as e:
        req["status"] = "Ditolak"
        print(e)
```

E. Fungsi Tanpa Parameter

```
# ==================== FUNGSI TANPA PARAMETER (ada return) ====================

def menu_utama():
    print("1. Login")
    print("2. Register")
    print("3. Keluar")
    try:
        return int(input("Pilih menu: "))
    except ValueError:
        return None


def ambil_id_terlihat():
    hasil = []
    for id_p in paket_data:
        if role_aktif == "admin" or paket_data[id_p]["pemilik"] == username_aktif:
            hasil.append(id_p)
    return hasil
```

F. Fungsi Dengan Parameter

```
# ==================== FUNGSI DENGAN PARAMETER (ada return) ====================

def ambil_jarak(asal, tujuan):
    return jarak_data[asal][tujuan]


def validasi_nama(nama):
    if nama == "":
        raise NamaPaketError("Nama paket tidak boleh kosong.")
    if len(nama) > 15:
        raise NamaPaketError("Nama paket tidak lebih dari 15 karakter.")
    return nama


def cari_paket_milik(id_paket):
    paket = paket_data[id_paket]
    if role_aktif != "admin" and paket["pemilik"] != username_aktif:
        raise KeyError(id_paket)
    return paket


def ambil_pilihan(judul, opsi, dilarang=None):
    while True:
        print(judul + ":")
        for n, opt in enumerate(opsi, 1):
            print(f"  {n}. {opt}")
        try:
            nomor = int(input("Pilih nomor: "))
            if nomor < 1 or nomor > len(opsi):
                raise IndexError
            pilihan = opsi[nomor - 1]
            if dilarang is not None and pilihan == dilarang:
                raise LokasiSamaError("Lokasi tujuan tidak boleh sama dengan lokasi asal.")
            return pilihan
        except ValueError:
            print("Input harus berupa angka.")
        except IndexError:
            print("Pilihan tidak valid.")
        except LokasiSamaError as e:
            print(e)
```

G. Fungsi Rekursif

```
def hitung_total_jarak(daftar_id, index):
    if index >= len(daftar_id):
        return 0
    jarak_ini = paket_data[daftar_id[index]]["jarak"]
    return jarak_ini + hitung_total_jarak(daftar_id, index + 1)
```

H. Fungsi Login

```
def proses_login():
    global username_aktif, role_aktif
    print("Login")
    username = input("Masukkan username: ").strip()
    password = input("Masukkan password: ").strip()
    try:
        if username == "" or password == "":
            raise AkunError("Username dan password tidak boleh kosong.")
        if username not in user_data:
            raise AkunError("Username salah.")
        if user_data[username]["password"] != password:
            raise AkunError("Password salah.")
    except AkunError as e:
        tampilkan_pesan(e, 1)
        bersihkan_layar()
        if input("Tidak punya akun, apakah ingin membuat akun? (Y/N): ").upper() == "Y":
            bersihkan_layar()
            proses_register()
        return False

    username_aktif = username
    role_aktif = user_data[username]["role"]
    tampilkan_pesan("Login berhasil!", 1)
    bersihkan_layar()
    tampilkan_pesan("Selamat datang di sistem kami!\n" + username, 1)
    return True
```

I. Dashboard dan Main

```
# ==================== DASHBOARD & MAIN ====================

def dashboard():
    global username_aktif, role_aktif
    aktif = True
    while aktif:
        bersihkan_layar()
        perbarui_jarak_semua()
        tampilkan_tabel_paket()

        print()
        if role_aktif == "admin":
            print("1. Tambah paket")
            print("2. Ubah paket")
            print("3. Hapus paket")
            print("4. Permintaan perubahan dari pelanggan")
        else:
            print("1. Ajukan perubahan paket")
            print("2. Riwayat permintaan saya")
        print("0. Logout")

        aksi = input("Pilih: ").strip()
        print()

        if role_aktif == "admin" and aksi == "1":
            tambah_paket()
```

```
        elif (role_aktif == "admin" and aksi == "2") or (role_aktif == "pengguna" and aksi ==
"1"):
            ubah_paket()
        elif role_aktif == "admin" and aksi == "3":
            hapus_paket()
        elif (role_aktif == "admin" and aksi == "4") or (role_aktif == "pengguna" and aksi ==
"2"):
            kelola_permintaan()
        elif aksi == "0":
            aktif = False
            username_aktif = ""
            role_aktif = ""
        else:
            tampilkan_pesan("Pilihan tidak ada.", 1)


def main():
    program = True
    bersihkan_layar()
    while program:
        pilih = menu_utama()
        if pilih is None:
            tampilkan_pesan("Invalid, masukkan angka", 1)
        elif pilih == 1:
            bersihkan_layar()
            if proses_login():
                dashboard()
        elif pilih == 2:
            bersihkan_layar()
            proses_register()
        elif pilih == 3:
            program = False
            print("Terima kasih!")
        else:
            tampilkan_pesan("Invalid, menu tidak ada", 1)
        if program:
            bersihkan_layar()


try:
    main()
except (KeyboardInterrupt, EOFError):
    print("\nProgram dihentikan oleh pengguna.")
finally:
    print("Sesi program selesai.")
```

4. Hasil Output

Tangkapan di bawah adalah keluaran teks asli program saat dijalankan (baris berisi masukan pengguna ditampilkan menyatu dengan prompt). Warna dan judul jendela terminal tidak ikut terekam.

```
1. Login
2. Register
3. Keluar
Pilih menu: abc
Invalid, masukkan angka
1. Login
2. Register
3. Keluar
Pilih menu: 3
Terima kasih!
Sesi program selesai.
```

Gambar 4.1 Menu utama saat masukan berupa huruf (“Invalid, masukkan angka”)

```
1. Login
2. Register
3. Keluar
Pilih menu: 9
Invalid, menu tidak ada
1. Login
2. Register
3. Keluar
Pilih menu: 3
Terima kasih!
Sesi program selesai.
```

Gambar 4.2 Menu utama saat angka di luar menu (“Invalid, menu tidak ada”)

```
1. Login
2. Register
3. Keluar
Pilih menu: 1
Login
Masukkan username: admin
Masukkan password: salah
Password salah.
Tidak punya akun, apakah ingin membuat akun? (Y/N): N
1. Login
2. Register
3. Keluar
Pilih menu: 3
Terima kasih!
Sesi program selesai.
```

Gambar 4.3 Login dengan password salah

```
1. Login
2. Register
3. Keluar
Pilih menu: 2
Register
Masukkan username: rafif
Masukkan password: rahasia123
Registrasi berhasil!
1. Login
2. Register
3. Keluar
Pilih menu: 3
Terima kasih!
Sesi program selesai.
```

Gambar 4.4 Register akun baru

```
Login sebagai: admin (admin)

DAFTAR SEMUA PAKET
ID      Pemilik   Paket           Layanan   Kurir   Asal        Tujuan      Jarak    Status
-------------------------------------------------------------------------------------------
1       pengguna  Laptop          Express   Budi    Balikpapan  Samarinda   115 km   Dikirim
2       pengguna  Dokumen         Express   Sinta   Balikpapan  Bontang     235 km   Dikirim
3       pengguna  Elektronik      Same Day  Andi    Samarinda   Tenggarong  35 km    Diproses
-------------------------------------------------------------------------------------------
Total jarak seluruh paket: 385 km
```

Gambar 4.5 Tabel paket admin beserta total jarak seluruh paket

```
TAMBAH PAKET
Username pelanggan: salahuser
Pelanggan tidak ditemukan.
```

Gambar 4.6 Tambah paket: username pelanggan tidak terdaftar

```
TAMBAH PAKET
Username pelanggan: pengguna
Nama paket (maks 15 karakter):
Nama paket tidak boleh kosong.
```

Gambar 4.7 Tambah paket: nama paket kosong

```
TAMBAH PAKET
Username pelanggan: pengguna
Nama paket (maks 15 karakter): Laptop Gaming ROG
Nama paket tidak lebih dari 15 karakter.
```

Gambar 4.8 Tambah paket: nama paket lebih dari 15 karakter

```
Jenis layanan:
  1. Reguler
  2. Express
  3. Same Day
Pilih nomor: 1
Kurir:
  1. Budi
  2. Sinta
  3. Andi
Pilih nomor: 1
Lokasi asal:
  1. Balikpapan
  2. Samarinda
  3. Bontang
  4. Tenggarong
  5. Penajam
Pilih nomor: 1
Lokasi tujuan:
  1. Balikpapan
  2. Samarinda
  3. Bontang
  4. Tenggarong
  5. Penajam
Pilih nomor: 1
Lokasi tujuan tidak boleh sama dengan lokasi asal.
```

Gambar 4.9 Tambah paket: nomor pilihan tidak valid dan tujuan sama dengan asal

```
Login sebagai: admin (admin)

DAFTAR SEMUA PAKET
ID      Pemilik   Paket           Layanan   Kurir   Asal        Tujuan      Jarak    Status
-------------------------------------------------------------------------------------------
1       pengguna  Laptop          Express   Budi    Balikpapan  Samarinda   115 km   Dikirim
2       pengguna  Kamera          Reguler   Budi    Balikpapan  Samarinda   115 km   Dikirim
-------------------------------------------------------------------------------------------
Total jarak seluruh paket: 230 km
```

Gambar 4.10 Tabel setelah paket baru berhasil ditambahkan

```
UBAH PAKET
Masukkan ID paket: 99
Paket tidak ditemukan.
```

Gambar 4.11 Ubah paket: ID tidak ditemukan

```
UBAH PAKET
Masukkan ID paket: 1
Data yang ingin diubah:
  1. Nama paket
  2. Jenis layanan
  3. Kurir
  4. Lokasi asal
  5. Lokasi tujuan
  6. Status
Pilih nomor: 9
Pilihan tidak valid.
```

Gambar 4.12 Ubah paket: nomor pilihan bukan angka dan di luar rentang

```
UBAH PAKET
Masukkan ID paket: 1
Data yang ingin diubah:
  1. Nama paket
  2. Jenis layanan
  3. Kurir
  4. Lokasi asal
  5. Lokasi tujuan
  6. Status
Pilih nomor: 1
Nama paket baru (maks 15 karakter):
Nama paket tidak boleh kosong.
```

Gambar 4.13 Ubah paket: nama paket baru kosong

```
UBAH PAKET
Masukkan ID paket: 1
Data yang ingin diubah:
  1. Nama paket
  2. Jenis layanan
  3. Kurir
  4. Lokasi asal
  5. Lokasi tujuan
  6. Status
Pilih nomor: 4
Lokasi asal baru:
  1. Balikpapan
  2. Samarinda
  3. Bontang
  4. Tenggarong
  5. Penajam
Pilih nomor: 2
Lokasi asal dan tujuan tidak boleh sama.
```

Gambar 4.14 Ubah paket: lokasi asal sama dengan tujuan

```
UBAH PAKET
Masukkan ID paket: 1
Data yang ingin diubah:
  1. Nama paket
  2. Jenis layanan
  3. Kurir
  4. Lokasi asal
  5. Lokasi tujuan
  6. Status
Pilih nomor: 2
Jenis layanan baru:
  1. Reguler
  2. Express
  3. Same Day
Pilih nomor: 1
Paket berhasil diubah! Jarak diperbarui otomatis.
```

Gambar 4.15 Ubah paket berhasil oleh admin

```
HAPUS PAKET
Masukkan ID paket: 1
Paket hanya bisa dihapus jika statusnya Sampai atau Dibatalkan.
```

Gambar 4.16 Hapus paket: status belum Sampai/Dibatalkan

```
HAPUS PAKET
Masukkan ID paket: 1
Yakin ingin menghapus? (Y/N): Y
Paket berhasil dihapus!
```

Gambar 4.17 Hapus paket berhasil

```
AJUKAN PERUBAHAN PAKET
Masukkan ID paket: 1
Data yang ingin diubah:
  1. Nama paket
  2. Jenis layanan
  3. Kurir
  4. Lokasi asal
  5. Lokasi tujuan
  6. Status
Pilih nomor: 2
Jenis layanan baru:
  1. Reguler
  2. Express
  3. Same Day
Pilih nomor: 1
Permintaan perubahan dikirim ke admin.
```

Gambar 4.18 Pengguna mengajukan perubahan paket

```
PERMINTAAN PERUBAHAN
ID      Pelanggan   Paket    Perubahan       Nilai baru    Status
-----------------------------------------------------------------
REQ001  pengguna    1        Jenis layanan   Reguler       Menunggu
```

Gambar 4.19 Riwayat permintaan milik pengguna

```
PERMINTAAN PERUBAHAN
ID      Pelanggan   Paket    Perubahan       Nilai baru    Status
-----------------------------------------------------------------
REQ001  pengguna    1        Jenis layanan   Reguler       Menunggu

ID permintaan yang diproses (kosongkan untuk kembali): REQ001
1. Setujui  2. Tolak : 1
Permintaan disetujui! Jarak diperbarui otomatis.
```

Gambar 4.20 Admin menyetujui permintaan perubahan

```
Login sebagai: admin (admin)

DAFTAR SEMUA PAKET
ID      Pemilik   Paket           Layanan   Kurir   Asal        Tujuan      Jarak    Status
-------------------------------------------------------------------------------------------
1       pengguna  Laptop          Reguler   Budi    Balikpapan  Samarinda   115 km   Dikirim
-------------------------------------------------------------------------------------------
Total jarak seluruh paket: 115 km
```

Gambar 4.21 Tabel setelah permintaan disetujui

5. Langkah-langkah Git

Berkas laporan dan kode program disimpan pada folder `posttest/posttest-apd-7/` di dalam repositori praktikum. Tiga perintah berikut dijalankan berurutan dari dalam folder tersebut.

5.1 Git Add

```
git add .
```

***Git add* memasukkan berkas yang baru dibuat atau diubah ke dalam *staging area*. Tahap ini belum menyimpan apa pun ke riwayat repositori, tetapi menandai berkas mana yang akan disertakan pada commit berikutnya.**

[tempel tangkapan layar *git add* di sini]

5.2 Git Commit

```
git commit -m "posttest 7: fungsi, prosedur, error handling, dan rekursi"
```

***Git commit* menyimpan perubahan yang sudah berada di *staging area* sebagai satu commit pada repositori lokal. Pesan commit menjelaskan isi perubahan, pada kasus ini penambahan fungsi dan prosedur, penanganan kesalahan, serta fungsi rekursif.**

[tempel tangkapan layar *git commit* di sini]

5.3 Git Push

```
git push origin main
```

***Git push* mengirimkan commit dari repositori lokal ke repositori jarak jauh sehingga perubahan dapat diakses melalui tautan repositori yang dikumpulkan.**

[tempel tangkapan layar *git push* di sini]